

APPLIED DATA STRUCTURES & ALGORITHMS LAB (BCS-553)

B.Tech CSE 5th SEMESTER

SUBMMITED TO:

MR. RASHAN RAJ

SUBMMITED BY:

MANKIRAT SINGH

ID: BCS2024542

ROLL NO: 2410303257

Lab Assignment 15: Subsets and Arrangements

Experiment No. 15.1 - All subsets of {1, 2, 3}

Experiment Title:

All subsets of {1, 2, 3}

Aim:

To print all subsets of an array using recursion and backtracking.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

Backtracking builds a solution one choice at a time. After a complete answer is printed, the last choice is undone so that the next choice can be tried. For subsets, each element is either skipped or included, giving 2^n subsets. For arrangements (permutations), a used[] array marks letters already placed, giving n! arrangements.

Algorithm / Procedure:

1. Keep a list current for the subset being built; i is the current index.
2. Base case: when i == length, print current in braces.
3. First skip a[i]: call subsets(a, i + 1).
4. Then include a[i]: add it, call subsets(a, i + 1), and remove it (backtrack).

Program / Queries / Commands:

```
// Name: Mankirat Singh
// ID: BCS2024542
// Roll No: 24103032571
// Assignment 15, Task 1
// Question: Print all subsets of {1, 2, 3} using recursion. At each
// index, first skip the value, then include it. At the end, print
// the subset in braces, for example {1, 3}, and print {} for the
// empty subset.

import java.util.ArrayList;

public class Main {
    static void subsets(int[] a, int i, ArrayList<Integer> current) {
        if (i == a.length) {
            System.out.print("{");
            for (int k = 0; k < current.size(); k++) {
                if (k > 0) {
                    System.out.print(", ");
                }
```

```
                System.out.print(current.get(k));
            }
            System.out.println("}");
            return;
        }

        // choice 1: skip a[i]
        subsets(a, i + 1, current);

        // choice 2: include a[i]
        current.add(a[i]);
        subsets(a, i + 1, current);

        // undo the choice
        current.remove(current.size() - 1);
    }

    public static void main(String[] args) {
        int[] a = {1, 2, 3};
        subsets(a, 0, new ArrayList<Integer>());
    }
}
```

Input:

Array {1, 2, 3}.

Output:

```
{}
{3}
{2}
{2, 3}
{1}
{1, 3}
{1, 2}
{1, 2, 3}
```

Time / Space Complexity: Time O(n * 2^n); Space O(n).

Result:

The program was compiled and executed successfully. All 8 subsets of {1, 2, 3} were printed, including the empty subset.

Viva-Voce Questions:

- How many subsets does a set of n elements have?
- What is backtracking?
- Why is the last element removed after the recursive call?

Experiment No. 15.2 - All arrangements of a, b, c

Experiment Title:

All arrangements of a, b, c

Aim:

To print all arrangements of three letters using a used[] array.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

Backtracking builds a solution one choice at a time. After a complete answer is printed, the last choice is undone so that the next choice can be tried. For subsets, each element is either skipped or included, giving 2^n subsets. For arrangements (permutations), a used[] array marks letters already placed, giving n! arrangements.

Algorithm / Procedure:

1. Keep a used[] array and a result[] array; pos is the position being filled.
2. Base case: when pos == length, print the arrangement.
3. For each letter that is not used: mark it used, place it at result[pos], recurse with pos + 1.
4. After the call returns, unmark the letter (backtrack).

Program / Queries / Commands:

```
// Name: Mankirat Singh
// ID: BCS2024542
// Roll No: 24103032571
// Assignment 15, Task 2
// Question: Print all arrangements of the letters a, b and c. Use a
// used[] array: pick a letter that is not used, mark it used, fill
// the next position, then unmark it. Print each arrangement on its
// own line.

public class Main {
    static void arrange(char[] letters, boolean[] used, char[] result,
            int pos) {
        if (pos == letters.length) {
            System.out.println(new String(result));
            return;
        }

        for (int i = 0; i < letters.length; i++) {
            if (!used[i]) {
                used[i] = true;
                result[pos] = letters[i];
                arrange(letters, used, result, pos + 1);
                used[i] = false;
            }
        }
    }
```

```
    public static void main(String[] args) {
        char[] letters = {'a', 'b', 'c'};
        boolean[] used = new boolean[letters.length];
        char[] result = new char[letters.length];
        arrange(letters, used, result, 0);
    }
}
```

Input:

Letters a, b, c.

Output:

```
abc
acb
bac
bca
cab
cba
```

Time / Space Complexity: Time O(n * n!); Space O(n).

Result:

The program was compiled and executed successfully. All 6 arrangements of a, b and c were printed.

Viva-Voce Questions:

- How many arrangements do n distinct letters have?
- What is the purpose of the used[] array?
- Why is used[i] set back to false?

Experiment No. 15.3 - All arrangements of x, y

Experiment Title:

All arrangements of x, y

Aim:

To print all arrangements of two letters using a used[] array.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

Backtracking builds a solution one choice at a time. After a complete answer is printed, the last choice is undone so that the next choice can be tried. For subsets, each element is either skipped or included, giving 2^n subsets. For arrangements (permutations), a used[] array marks letters already placed, giving n! arrangements.

Algorithm / Procedure:

1. Keep a used[] array and a result[] array; pos is the position being filled.
2. Base case: when pos == length, print the arrangement.
3. For each letter that is not used: mark it used, place it at result[pos], recurse with pos + 1.
4. After the call returns, unmark the letter (backtrack).

Program / Queries / Commands:

```
// Name: Mankirat Singh
// ID: BCS2024542
// Roll No: 24103032571
// Assignment 15, Task 3
// Question: Print all arrangements of the letters x and y, one per
// line, using a used[] array: pick a letter that is not used, mark
// it used, fill the next position, then unmark it.

public class Main {
    static void arrange(char[] letters, boolean[] used, char[] result,
            int pos) {
        if (pos == letters.length) {
            System.out.println(new String(result));
            return;
        }

        for (int i = 0; i < letters.length; i++) {
            if (!used[i]) {
                used[i] = true;
                result[pos] = letters[i];
                arrange(letters, used, result, pos + 1);
                used[i] = false;
            }
        }
    }
```

```
    public static void main(String[] args) {
        char[] letters = {'x', 'y'};
        boolean[] used = new boolean[letters.length];
        char[] result = new char[letters.length];
        arrange(letters, used, result, 0);
    }
}
```

Input:

Letters x, y.

Output:

```
xy
yx
```

Time / Space Complexity: Time O(n * n!); Space O(n).

Result:

The program was compiled and executed successfully. Both arrangements of x and y were printed.

Viva-Voce Questions:

- How many arrangements exist for 2 letters?
- Is this recursion or iteration?
- How does the program change for more letters?